



Exercise: Linear Regression (Vectorization)

Nguyễn Quốc Thái

Hồ Quang Hiện

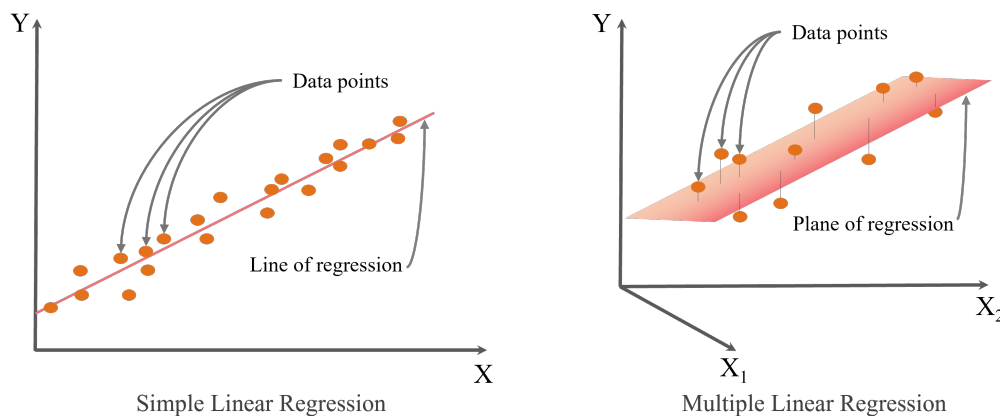
Đình Quang Vinh

I. Giới thiệu

Bạn mở một quán cà phê nhỏ và nhận ra: những ngày nắng nóng, số ly cold brew bán ra tăng vọt; trời mát thì khách chuộng latte nóng và doanh thu giảm nhẹ. Bạn bắt đầu ghi lại dữ liệu mỗi ngày: nhiệt độ trung bình, số ly bán ra, doanh thu. Sau vài tuần, bạn tự hỏi: "Nếu mai nhiệt độ 34°C, mình sẽ bán khoảng bao nhiêu ly?"

Đó chính là lúc các thuật toán bước vào đời sống. Khi một đại lượng (doanh số) thay đổi theo một đại lượng khác (nhiệt độ), ta tìm một đường thẳng ‘phù hợp nhất’ đi qua các điểm dữ liệu để mô tả mối quan hệ ấy. Với *linear regression* (hồi quy tuyến tính), đường thẳng $y = wx + b$ cho ta hai ý nghĩa:

- **w**: mỗi khi nhiệt độ tăng 1°C, doanh số ước tính tăng/giảm bao nhiêu.
- **b**: doanh số kỳ vọng khi nhiệt độ bằng 0.



Hình 1: Hình ảnh minh họa về Linear Regression.

Từ đường thẳng này, ta có thể dự đoán tương lai (forecast), giải thích tác động (diễn giải hệ số), và ra quyết định (nhập hàng, bố trí nhân sự). Tất nhiên, không phải lúc nào mọi thứ cũng

"tuyến tính": có nhiều, ngoại lệ, và nhiều yếu tố khác (thứ trong tuần, khuyến mãi). Nhưng bắt đầu bằng hồi quy tuyến tính giúp ta có một mô hình đủ đơn giản, minh bạch để giải quyết các vấn đề thường gặp.

Trong phần tiếp theo chúng ta sẽ tìm hiểu về các phương pháp tối ưu cho huấn luyện các mô hình hồi quy tuyến tính.

Mục lục

I.	Giới thiệu	1
II.	Mô hình Linear Regression	5
II.1.	Tổng quan	5
II.2.	Linear Regression áp dụng từng mẫu dữ liệu	5
II.3.	Linear Regression cho batch (mini-batch hoặc full batch)	6
III.	Xây dựng mô hình Linear Regression cho bài toán advertising	7
IV.	Xây dựng mô hình Linear Regression cho bài toán dự đoán Bitcoin	14
V.	Câu hỏi trắc nghiệm	24
	Phụ lục	26

Để việc đọc thuận tiện hơn, bảng sau liệt kê và chuẩn hoá các ký hiệu sẽ xuất hiện xuyên suốt nội dung bài.

Bảng Ký hiệu Toán học

Ký hiệu	Ý nghĩa
x_i	Đặc trưng (đầu vào) của mẫu thứ i (trường hợp 1 biến).
y_i	Nhãn (giá trị mục tiêu) của mẫu i .
$\hat{y}_i$	Giá trị dự đoán từ x_i .
w, b	Trọng số và bias của mô hình.
ε_i	Sai số ngẫu nhiên (nhiều) giữa y_i và $\hat{y}_i$.
d	Số đặc trưng.
m	Kích thước batch (số mẫu trong một batch).
$\mathbb{R}^{a \times b}$	Không gian các ma trận số thực kích thước $a \times b$ (a hàng, b cột).
$\mathbf{x} \in \mathbb{R}^{(d+1)}$	Vector đặc trưng đã gộp bias cho <i>một</i> mẫu.
$\boldsymbol{\theta} \in \mathbb{R}^{(d+1)}$	Vector tham số (gồm các w_j và b).
$\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$	Ma trận đặc trưng của một batch m mẫu (đã gộp bias).
$\mathbf{y} \in \mathbb{R}^m$	Vector mục tiêu của batch.
L	Hàm mất mát: thước đo độ sai lệch giữa dự đoán và thực tế.
$\frac{\partial L_i}{\partial w}, \frac{\partial L_i}{\partial b}$	Thành phần gradient theo từng tham số tại mẫu i .
$\nabla_{\boldsymbol{\theta}} L$	Gradient: xác định hướng và tốc độ dốc nhất để giảm lỗi.
η	Tốc độ học (learning rate).

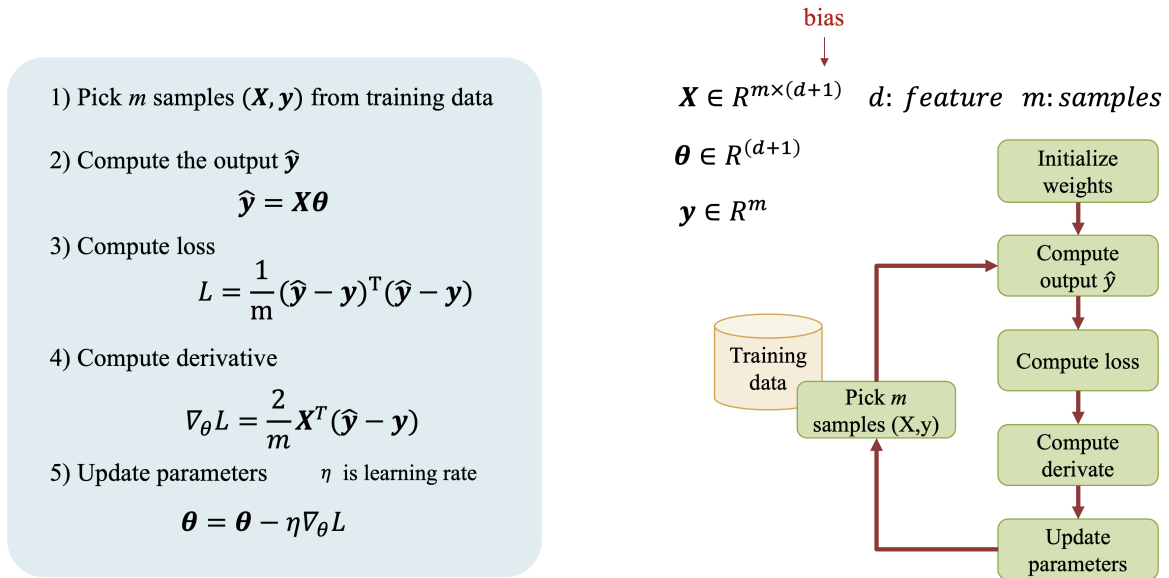
II. Mô hình Linear Regression

II.1. Tổng quan

Linear Regression (hồi quy tuyến tính) là phương pháp cổ điển bắt nguồn từ phương pháp “bình phương tối thiểu” do *Legendre* (1805) và *Gauss* (1809) phát triển; thuật ngữ *regression* được *Francis Galton* (1886) đặt khi mô tả hiện tượng “hồi quy về trung bình” trong nghiên cứu di truyền.

Mục tiêu của Linear Regression là mô hình hoá quan hệ gần tuyến tính giữa đầu vào và đầu ra để dự đoán hoặc giải thích ảnh hưởng của từng biến đầu vào lên kết quả.

Trong bài toán có d đặc trưng x và nhãn y áp dụng với từng mẫu dữ liệu, mô hình:



Hình 2: Hình ảnh minh họa về quá trình huấn luyện Linear Regression với m samples.

II.2. Linear Regression áp dụng từng mẫu dữ liệu

Step 1: Khởi tạo

Khởi tạo tham số đã gộp bias:

$$\theta = \begin{bmatrix} w_1 \\ \vdots \\ w_d \\ b \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad \theta \sim \mathcal{N}(0, 0.01^2 I_{d+1}).$$

Chú ý: Chúng ta hoàn toàn có thể đặt biến b đứng trước w_1 trong vector θ , thay vì đứng sau w_d như trên. Lúc này, vector x ở bước sau cũng cần thay đổi vị trí số 1 tương ứng.

Step 2: Lấy một mẫu

Chọn một cặp (x, y) với

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \\ 1 \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad y \in \mathbb{R}.$$

Step 3: Dự đoán (vector hoá)

$$\hat{y} = \mathbf{x}^T \boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{x} \in \mathbb{R}.$$

Step 4: Mất mát từng mẫu (MSE)

$$L = (\hat{y} - y)^2.$$

Step 5: Gradient theo vector tham số

$$\nabla_{\boldsymbol{\theta}} L = 2\mathbf{x}(\hat{y} - y) \in \mathbb{R}^{(d+1)}.$$

Step 6: Cập nhật SGD (dùng 1 mẫu mỗi bước, learning rate α)

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} L$$

Lặp lại từ **Step 2** qua các mẫu cho đến khi hội tụ hoặc đạt số vòng lặp tối đa.

II.3. Linear Regression cho batch (mini-batch hoặc full batch)

Step 1: Khởi tạo

Khởi tạo tham số đã gộp bias:

$$\boldsymbol{\theta} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \\ b \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad \boldsymbol{\theta} \sim \mathcal{N}(0, 0.01^2 I_{d+1}).$$

Step 2: Lấy một batch

Chọn batch m mẫu $(X^{(i)}, y^{(i)})$ với $i = 1, \dots, m$ và ghép thành:

$$\mathbf{X} = \begin{bmatrix} X^{(1)} \\ \vdots \\ X^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times (d+1)}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix} \in \mathbb{R}^m,$$

trong đó mỗi $X^{(i)} \in \mathbb{R}^{1 \times (d+1)}$ là một vector hàng.

Step 3: Dự đoán cho batch

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} \in \mathbb{R}^m.$$

Step 4: Mất mát cho batch

$$L = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \frac{1}{m} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2.$$

Step 5: Gradient cho batch

$$\nabla_{\boldsymbol{\theta}} L = \frac{2}{m} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y}) \in \mathbb{R}^{(d+1)}.$$

Step 6: Cập nhật tham số

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L.$$

Lặp lại từ Step 2 đến khi hội tụ hoặc đạt số epoch tối đa.

III. Xây dựng mô hình Linear Regression cho bài toán advertising

Ở bài toán này chúng ta sẽ thực hành các bài tập code về giải thuật linear regression cho bài toán advertising theo cách vectorization dùng stochastic gradient descent, m samples (mini-batch gradient descent), and N samples (batch gradient descent).

Thực hiện train linear regression model trên tập data advertising.csv theo các yêu cầu sau. Các bạn sẽ dựa trên 3 thông tin đầu vào là TV, Radio, Newspaper để dự đoán Sale.

Giới thiệu về tập data: Data có 200 samples (rows), gồm 4 cột thông tin Tv, Radio, Newspaper, và Sales. Đề bài yêu cầu dùng thông tin ở 3 cột đầu tiên (Tv, Radio, Newspaper) để dự đoán được cột cuối cùng (Sale) dùng linear regression model. Code tải dataset được cung cấp ở bên dưới như sau:

```
1 !gdown 1x30X304P0-bonyooK4FKcRhYrTmT4kif
2 !unzip advertising.zip
```

Để chuẩn hoá data đầu vào, bạn được cung cấp trước function đọc dữ liệu và chuẩn hoá `mean_normalization(X)` như bên dưới:

mean_normalization(X)

```
1 import numpy as np
2 from numpy import genfromtxt
```

```

3 import matplotlib.pyplot as plt
4
5 # dataset
6 data = genfromtxt("../content/advertising.csv", delimiter=",", skip_header=1)
7 N = data.shape[0]
8 X = data[:, :3]
9 y = data[:, 3:]
10
11 # Normalize input data by using mean normalization
12 def mean_normalization(X):
13     N = len(X)
14     maxi = np.max(X)
15     mini = np.min(X)
16     avg = np.mean(X)
17     X = (X-avg) / (maxi-mini)
18     X_b = np.c_[np.ones((N, 1)), X]
19     return X_b, maxi, mini, avg
20
21 X_b, maxi, mini, avg = mean_normalization(X)

```

Yêu cầu của bài tập này là các bạn lần lượt hiện thực lại giải thuật linear regression để dự đoán Sales dựa vào các yêu cầu sau:

Hoàn thành function **stochastic_gradient_descent()** để huấn luyện data sử dụng Stochastic Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (4 inputs) X_b , y , n_epochs , $learning_rate$
- **output:** θ_{path} , $losses$

stochastic_gradient_descent()

```

1 def stochastic_gradient_descent(X_b, y, n_epochs=50, learning_rate=0.01):
2     N, d_plus1 = X_b.shape
3
4     # Step 1: Init parameters
5     thetas = np.asarray([[1.16270837], [-0.81960489],
6                          [1.39501033], [0.29763545]])
7     thetas_path = [thetas.copy()]
8     losses = []
9     # Step 2-6: SGD loop
10    for epoch in range(n_epochs):
11        for i in range(N):
12            random_index = i
13            xi = X_b[random_index:random_index+1]
14            yi = y[random_index:random_index+1]
15
16            # Step 3: prediction
17            ***Your code here***
18
19            # Step 4: MSE loss for 1 sample
20            ***Your code here***
21

```

```

22     # Step 5: gradient
23     ***Your code here***
24
25     # Step 6: update
26     ***Your code here***
27
28     # log
29     thetas_path.append(thetas.copy())
30     losses.append(li[0][0])
31     return thetas_path, losses

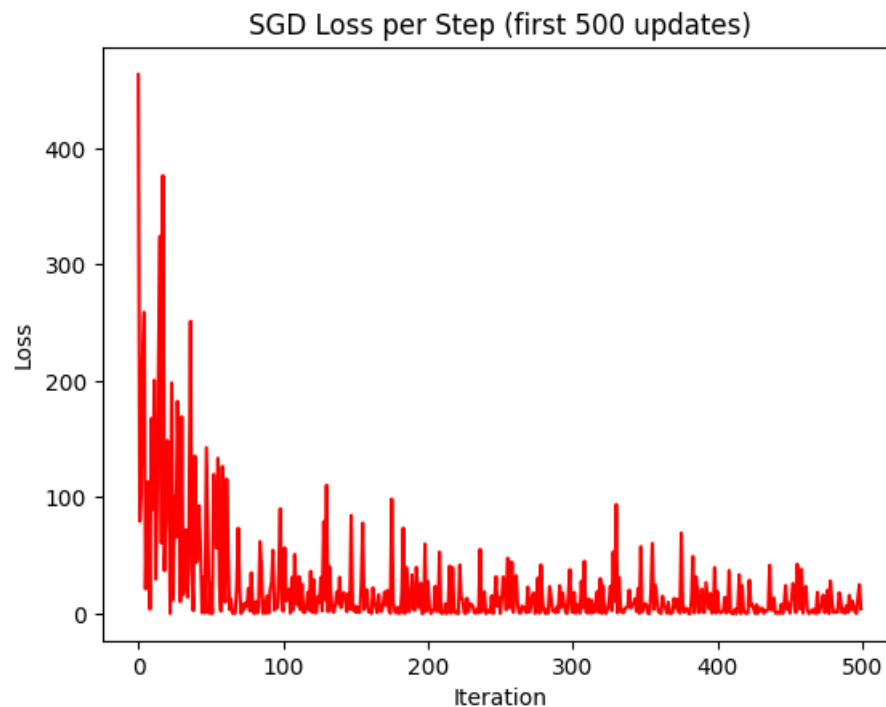
```

Hình 2 là kết quả sau khi thực thi đoạn code sau:

```

1 sgd_theta, losses = stochastic_gradient_descent(X_b, y, n_epochs=50, learning_rate=0.01
2
3 # In loss cho 500 bước đầu
4 x_axis = list(range(500))
5 plt.plot(x_axis, losses[:500], color="r")
6 plt.xlabel("Iteration")
7 plt.ylabel("Loss")
8 plt.title("SGD Loss per Step (first 500 updates)")
9 plt.show()

```



Hình 3: Kết quả loss values sử dụng Stochastic Gradient Descent

Hoàn thành function `mini_batch_gradient_descent()` để huấn luyện data sử dụng Mini-batch Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (5 inputs) `X_b`, `y`, `n_epochs`, `minibatch_size`, `learning_rate`
- **output:** `thetas_path`, `losses`

`mini_batch_gradient_descent()`

```

1 def mini_batch_gradient_descent(X_b, y, n_epochs=50, minibatch_size=20, learning_rate=0
                                .01):
2     N, d_plus1 = X_b.shape
3     # Step 1: Init parameters
4     thetas = np.asarray([[1.16270837], [-0.81960489],
5                           [1.39501033], [0.29763545]])
6     thetas_path = [thetas.copy()]
7     losses = []
8     for epoch in range(n_epochs):
9         # shuffle data
10        shuffled_indices = np.random.permutation(N)
11        X_b_shuffled = X_b[shuffled_indices]
12        y_shuffled = y[shuffled_indices]
13
14        for i in range(0, N, minibatch_size):
15            xi = X_b_shuffled[i:i+minibatch_size]    # (m, d+1)
16            yi = y_shuffled[i:i+minibatch_size]      # (m, 1)
17
18            # Step 3: prediction
19            ***Your code here***
20
21            # Step 4: MSE loss for minibatch
22            ***Your code here***
23
24            # Step 5: gradient for minibatch
25            ***Your code here***
26
27            # Step 6: update
28            ***Your code here***
29
30            # log
31            thetas_path.append(thetas.copy())
32            losses.append(loss)
33    return thetas_path, losses

```

Hình 3 là kết quả sau khi thực thi đoạn code sau:

```

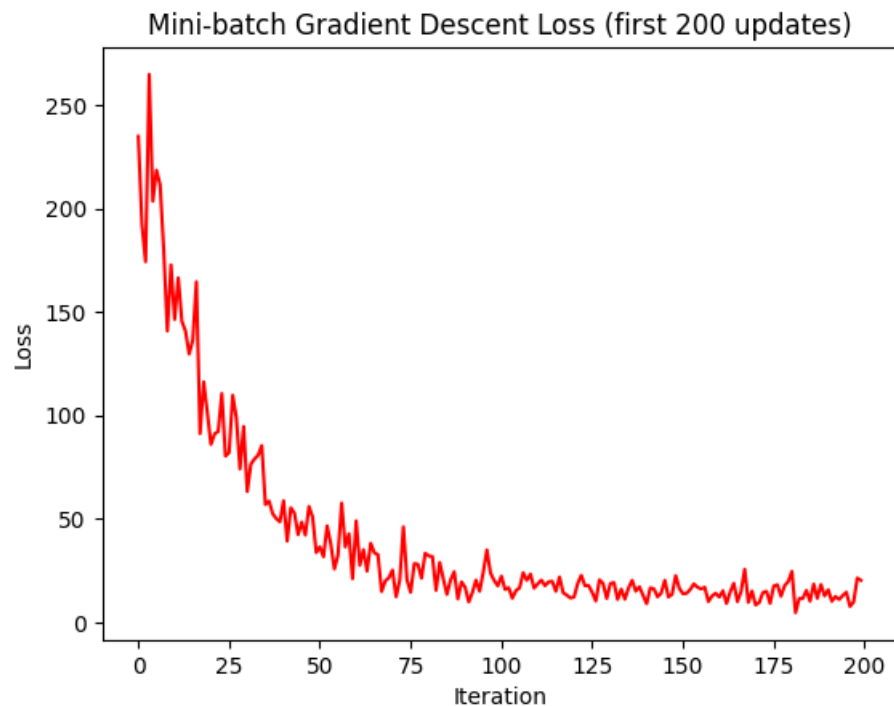
1 mbgd_thetas, losses = mini_batch_gradient_descent(X_b, y, n_epochs=50, minibatch_size=
                                20, learning_rate=0.01)
2
3 # visualize
4 x_axis = list(range(200))

```

```

5 plt.plot(x_axis, losses[:200], color="r")
6 plt.xlabel("Iteration")
7 plt.ylabel("Loss")
8 plt.title("Mini-batch Gradient Descent Loss (first 200 updates)")
9 plt.show()

```



Hình 4: Kết quả loss values sử dụng Mini batch Gradient Descent

Hoàn thành function **batch_gradient_descent()** để huấn luyện data sử dụng batch Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (4 inputs) X_b , y , n_epochs , $learning_rate$
- **output:** θ_{path} , $losses$

batch_gradient_descent()

```

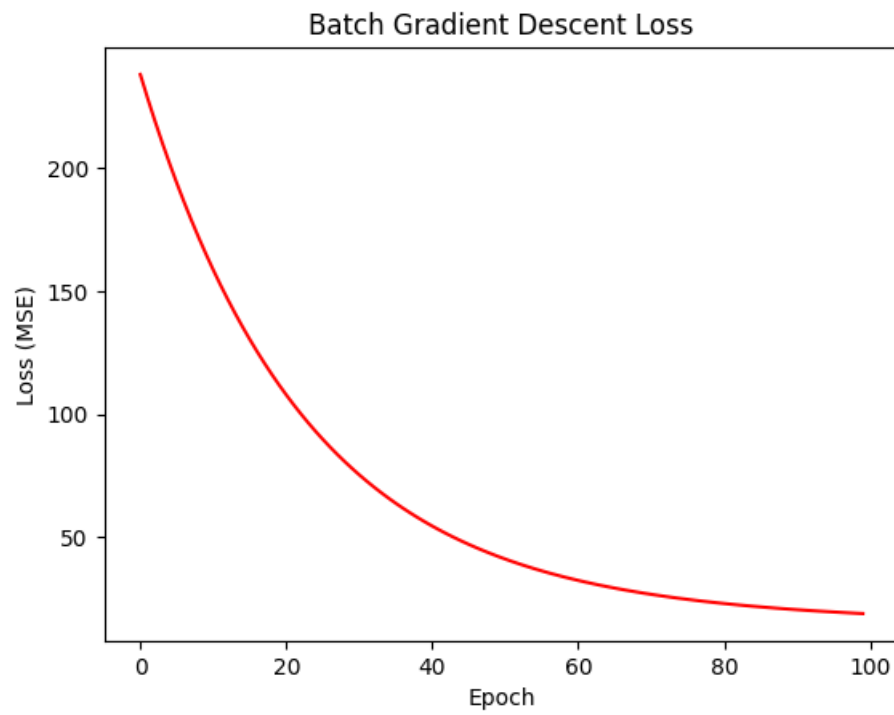
1 def batch_gradient_descent(X_b, y, n_epochs=100, learning_rate=0.01):
2     N, d_plus1 = X_b.shape
3     # Step 1: Init parameters
4     thetas = np.asarray([[1.16270837], [-0.81960489], [1.39501033], [0.29763545]])
5     thetas_path = [thetas.copy()]
6     losses = []
7
8     for epoch in range(n_epochs):
9         # Step 3: prediction
10        ***Your code here***

```

```
11
12     # Step 4: MSE loss
13     ***Your code here***
14
15     # Step 5: gradient
16     ***Your code here***
17
18     # Step 6: update parameters
19     ***Your code here***
20
21     # log
22     thetas_path.append(thetas.copy())
23     losses.append(loss)
24
25     return thetas_path, losses
```

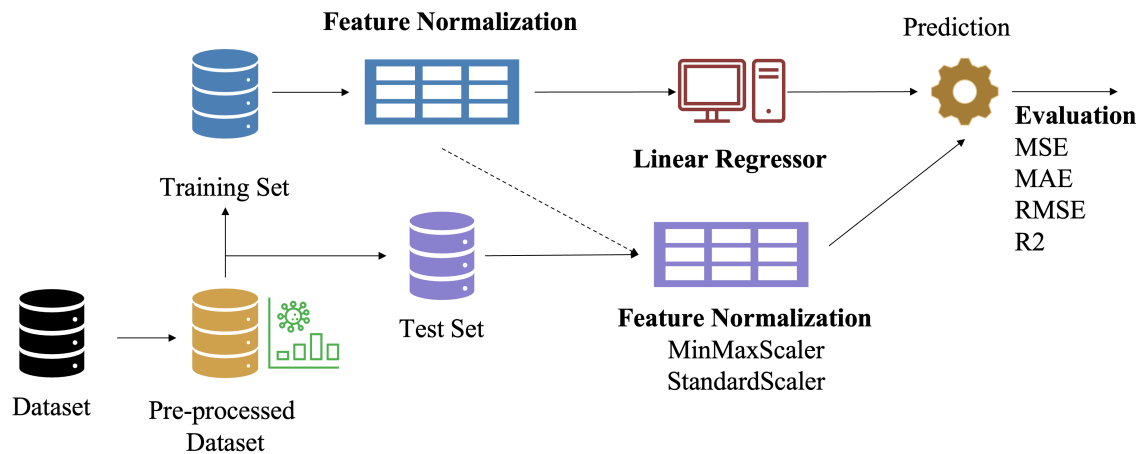
Hình 4 là kết quả sau khi thực thi đoạn code sau:

```
1 bgd_thetas, losses = batch_gradient_descent(X_b, y, n_epochs=100, learning_rate=0.01)
2
3 # visualize
4 x_axis = list(range(100))
5 plt.plot(x_axis, losses[:100], color="r")
6 plt.xlabel("Epoch")
7 plt.ylabel("Loss (MSE)")
8 plt.title("Batch Gradient Descent Loss")
9 plt.show()
```



Hình 5: Kết quả loss values sử dụng Batch Gradient Descent

IV. Xây dựng mô hình Linear Regression cho bài toán dự đoán Bitcoin



Hình 6: Các bước thực hiện xây dựng mô hình hồi quy tuyến tính cho bộ dữ liệu bitcoin.

Bước 1: Tải và đọc dữ liệu

Đầu tiên, ta tiến hành tải bộ dữ liệu Bitcoin forecasting bằng code sau:

```
1 !gdown 1mc89F1dkQ6XRQnwQTqqryjaXt5ZWXTT0
2 !unzip BTC-Daily.zip
```

Import các thư viện cần thiết và tiến hành đọc file.csv như sau:

```
1 import pandas as pd
2 import numpy as np
3 import datetime as dt
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv("../content/BTC-Daily.csv")
7 df = df.drop_duplicates()
8 df.head()
```

Ở bước này ta sử dụng `pandas` để đọc dữ liệu lịch sử giá Bitcoin từ file `BTC-Daily.csv`. Lệnh `drop_duplicates()` giúp loại bỏ các dòng trùng lặp, đảm bảo dữ liệu sạch. Cuối cùng, `df.head()` hiển thị 5 dòng đầu tiên của bảng dữ liệu, cho phép ta kiểm tra nhanh cấu trúc và các cột trong dataset.

	unix	date	symbol	open	high	low	close	Volume BTC	Volume USD
0	1646092800	2022-03-01 00:00:00	BTC/USD	43221.71	43626.49	43185.48	43185.48	49.006289	2.116360e+06
1	1646006400	2022-02-28 00:00:00	BTC/USD	37717.10	44256.08	37468.99	43178.98	3160.618070	1.364723e+08
2	1645920000	2022-02-27 00:00:00	BTC/USD	39146.66	39886.92	37015.74	37712.68	1701.817043	6.418008e+07
3	1645833600	2022-02-26 00:00:00	BTC/USD	39242.64	40330.99	38600.00	39146.66	912.724087	3.573010e+07
4	1645747200	2022-02-25 00:00:00	BTC/USD	38360.93	39727.97	38027.61	39231.64	2202.851827	8.642149e+07

Hình 7: Vài mẫu dữ liệu của dataset Bitcoin forecasting

Bước 2: Khám phá dữ liệu

```

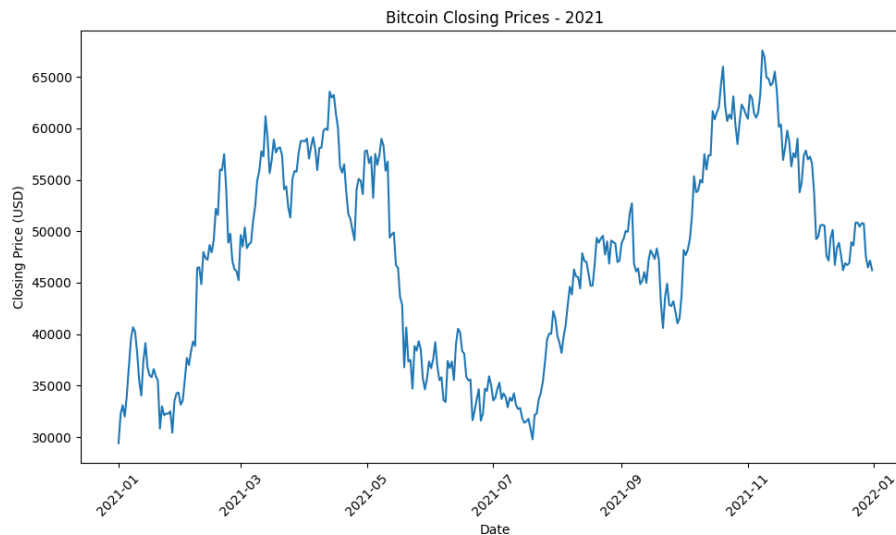
1  # Range of dates covered
2  df["date"] = pd.to_datetime(df["date"])
3  date_range = str(df["date"].dt.date.min()) + " to " + str(df["date"].dt.date.max())
4  print(date_range)
5
6  df["year"] = df["date"].dt.year
7  df["month"] = df["date"].dt.month
8  df["day"] = df["date"].dt.day
9
10
11 unique_years = df["year"].unique()
12 for year in unique_years:
13
14     dates = pd.date_range(start=f"{year}-01-01", end=f"{year}-12-31", freq="D")
15     year_month_day = pd.DataFrame({"date": dates})
16     year_month_day["year"] = year_month_day["date"].dt.year
17     year_month_day["month"] = year_month_day["date"].dt.month
18     year_month_day["day"] = year_month_day["date"].dt.day
19
20
21 merged_data = pd.merge(year_month_day, df, on=["year", "month", "day"], how="left")
22
23 # Plot
24 plt.figure(figsize=(10, 6))
25 plt.plot(merged_data["date_x"], merged_data["close"])
26 plt.title(f"Bitcoin Closing Prices - {year}")
27 plt.xlabel("Date")
28 plt.ylabel("Closing Price (USD)")
29 plt.xticks(rotation=45)
30 plt.tight_layout()
31 plt.show()

```

Trong bước này, dữ liệu được xử lý và trực quan hoá theo các bước:

- Chuyển đổi dữ liệu về dạng ngày tháng để xác định khoảng thời gian mà tập dữ liệu bao phủ.

- Tách thêm các cột `year`, `month`, `day` nhằm dễ dàng thao tác theo từng năm.
- Với mỗi năm, dữ liệu được ghép với dãy ngày đầy đủ để tránh bị thiếu ngày (nếu thị trường nghỉ giao dịch hoặc dữ liệu không đủ).
- Vẽ biểu đồ giá đóng cửa (`close`) theo thời gian cho từng năm, giúp quan sát trực quan biến động giá Bitcoin và phát hiện các xu hướng chính trong từng giai đoạn.



Hình 8: Biểu đồ giá kết thúc phiên trong năm 2021

Bước 3: Tạo biểu đồ nền giá giao dịch từ năm 2019 - 2022

Cài đặt thư viện `mplfinance` để vẽ biểu đồ nền giao dịch

```
1 !pip install mplfinance
```

Tiếp theo, ta thực thi code bên dưới để tạo biểu đồ giao dịch từ năm 2019 - 2022

```
1 import matplotlib.pyplot as plt
2 import matplotlib.dates as mdates
3 from mplfinance.original_flavor import candlestick_ohlc
4 import datetime
5
6 # Filter data for 2019-2022
7 df_filtered = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2022-12-31")]
8
9 # Convert date to matplotlib format
10 df_filtered = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2022-12-31")].copy()
11 df_filtered["date"] = df_filtered["date"].map(mdates.date2num)
12
13
```

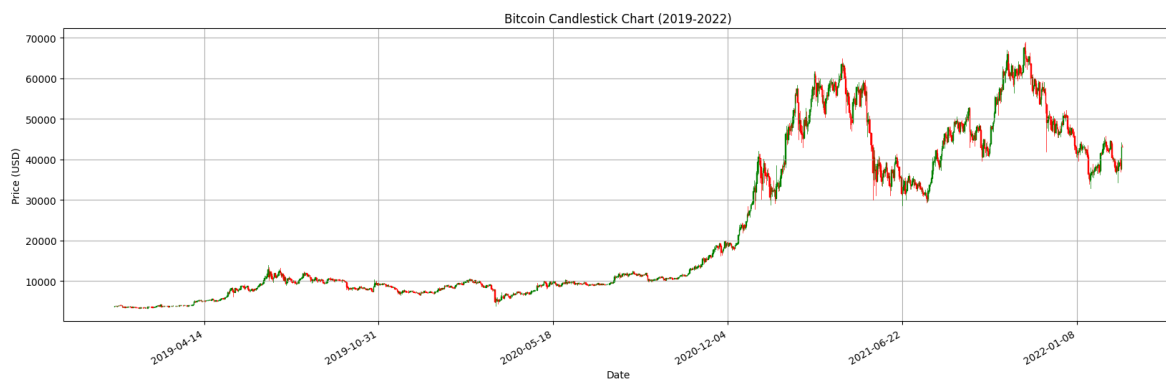
```

14 # Create the candlestick chart
15 fig, ax = plt.subplots(figsize=(20, 6))
16
17 candlestick_ohlc(ax, df_filtered[["date", "open", "high", "low", "close"]].values,
18                      width=0.6, colorup="g", colordown="r")
19
20 ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
21 fig.autofmt_xdate()
22
23 plt.title("Bitcoin Candlestick Chart (2019-2022)")
24 plt.xlabel("Date")
25 plt.ylabel("Price (USD)")
26 plt.grid(True)
27
28 # Save the plot as a PDF
29 plt.savefig("bitcoin_candlestick_2019_2022.pdf")
30 plt.show()

```

Trong bước này, dữ liệu được lọc cho giai đoạn từ năm 2019 đến 2022 và biểu diễn dưới dạng biểu đồ nến (candlestick).

- Mỗi nến thể hiện biến động giá trong một ngày giao dịch, gồm 4 giá trị: **open**, **high**, **low**, **close**.
- Nến màu xanh (green) biểu thị giá đóng cửa cao hơn giá mở cửa, trong khi nến màu đỏ (red) thể hiện chiều ngược lại.
- Biểu đồ nến cho phép quan sát trực quan xu hướng tăng/giảm trong ngắn hạn và các biến động mạnh của giá Bitcoin trong từng giai đoạn.
- Kết quả cuối cùng được lưu ra file PDF để thuận tiện cho việc sử dụng trong báo cáo.



Hình 9: Biểu đồ giao dịch từ năm 2019 - 2022

Bước 4: Tạo mô hình Linear Regression

Để huấn luyện mô hình Linear Regression, ta cần xây dựng các hàm cơ bản như `predict`, `gradient` và `update_weight`.

```
1 def predict(X, w, b):
2     ***Your code here***
3
4 def gradient(y_hat, y, x):
5     ***Your code here***
6     return (dw,db, cost)
7
8 def update_weight(w,b,lr,dw,db):
9     ***Your code here***
10    return (w_new, b_new)
```

Bước 5: Chia tập train và test

```
1 from sklearn.preprocessing import StandardScaler
2 from sklearn.model_selection import train_test_split
3
4 X = df[["open", "low", "high"]].values
5 y = df["close"].values
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.3, random_state=42, shuffle=True
9 )
10
11 scalar = StandardScaler()
12
13 X_train_scaled = scalar.fit_transform(X_train)
14 X_test_scaled = scalar.transform(X_test)
```

Tách dữ liệu:

- X : các đặc trưng open, low, high.
- y : nhãn close.

Chia dữ liệu bằng `train_test_split` (70% train, 30% test, `random_state=42`).

Chuẩn hóa dữ liệu với `StandardScaler`:

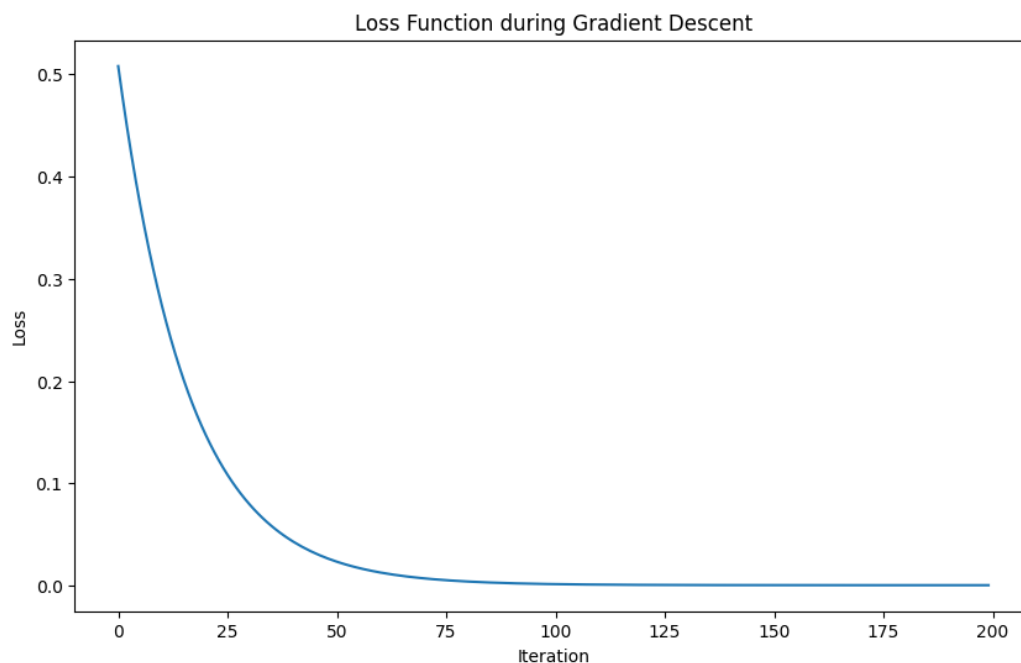
- `fit_transform` trên tập train.
- `transform` trên tập test.

Bước 6: Huấn luyện và đánh giá mô hình tự xây dựng

```
1 def linear_regression_vectorized(X, y, learning_rate=0.01, num_iterations=200):
2     n_samples, n_features = X.shape
3     w = np.zeros(n_features) # Initialize weights
4     b = 0 # Initialize bias
5     losses = []
6
7     ***Your code here***
8
9     return w, b, losses
```

Huấn luyện mô hình bằng Batch Gradient Descent và trực quan hoá quá trình tối ưu bằng cách vẽ biểu đồ hàm mất mát theo số vòng lặp để quan sát mức độ hội tụ của mô hình.

```
1 w, b, losses = linear_regression_vectorized(X_train.values, y_train.values,
2                                             learning_rate=0.01 , num_iterations=200)
3
4 # Plot the loss function
5 plt.figure(figsize=(10, 6))
6 plt.plot(losses)
7 plt.xlabel("Iteration")
8 plt.ylabel("Loss")
9 plt.title("Loss Function during Gradient Descent")
10 plt.show()
```



Hình 10: Biểu đồ hàm loss trong quá trình huấn luyện

Sau khi mô hình được huấn luyện, ta tiến hành đánh giá hiệu quả dự đoán bằng cách tính các chỉ số sai số phổ biến như **RMSE**, **MAE**, **MAPE**, và hệ số xác định R^2 cho cả tập huấn luyện và tập kiểm tra. Các chỉ số này giúp kiểm chứng độ chính xác và khả năng khái quát hoá của mô hình.

```

1 from sklearn.metrics import r2_score
2
3 # Make predictions on the test set
4 y_pred = predict(X_test, w, b)
5
6 # Calculate RMSE
7 rmse = np.sqrt(np.mean((y_pred - y_test) ** 2))
8
9 # Calculate MAE
10 mae = np.mean(np.abs(y_pred - y_test))
11
12 # Calculate MAPE
13 mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
14
15
16 # Calculate R-squared on training data
17 y_train_pred = predict(X_train, w, b)
18 train_accuracy = r2_score(y_train, y_train_pred)
19
20 # Calculate R-squared on testing data
21 test_accuracy = r2_score(y_test, y_pred)
22
23 print("Root Mean Square Error (RMSE):", round(rmse, 4))
24 print("Mean Absolute Error (MAE):", round(mae, 4))
25 print("Training Accuracy (R-squared):", round(train_accuracy, 4))
26 print("Testing Accuracy (R-squared):", round(test_accuracy, 4))

```

Để minh hoạ kết quả dự đoán, ta thử nghiệm trên dữ liệu quý 1 năm 2019 (từ 01/01 đến 31/03). Biểu đồ dưới đây thể hiện sự so sánh giữa **giá thực tế** và **giá dự đoán**, giúp quan sát trực quan mức độ sai lệch và xu hướng giữa hai đường giá.

```

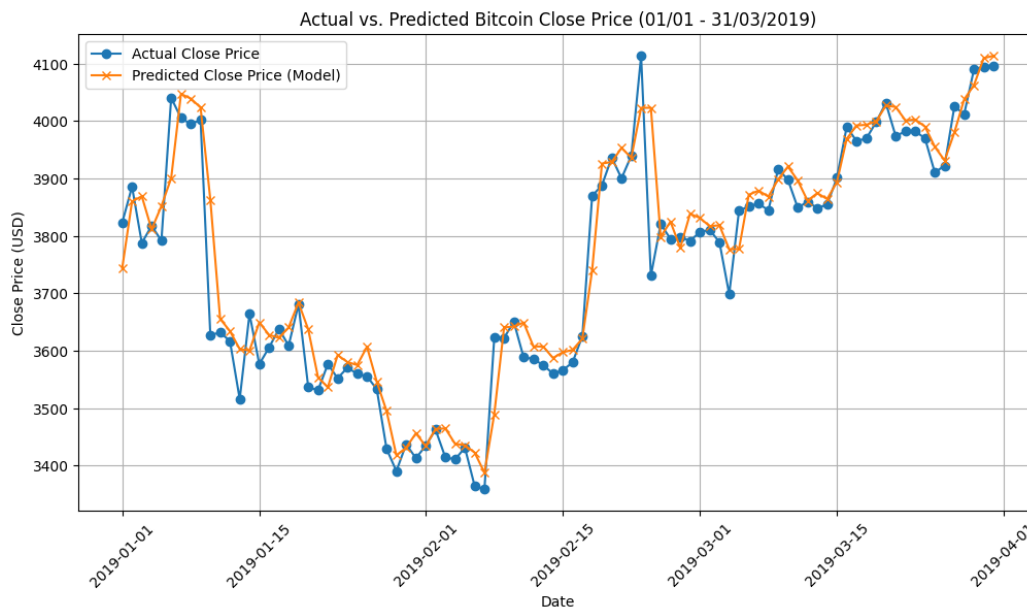
1 # Lọc dữ liệu 2019 Q1
2 df_2019_Q1 = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2019-03-31")].copy()
3
4 # Chọn đúng các cột feature đã train
5 X_2019 = df_2019_Q1[["open", "low", "high"]].values
6 X_2019_scaled = scalar.transform(X_2019) # dùng scalar đã fit trước đó
7 y_2019 = df_2019_Q1["close"].values
8
9 # Dự đoán bằng mô hình
10 y_pred_2019 = predict(X_2019_scaled, w, b)
11
12 # Gắn vào dataframe để dễ vẽ
13 df_2019_Q1["predicted_close"] = y_pred_2019
14

```

```

15 # Vẽ biểu đồ
16 plt.figure(figsize=(12,6))
17 plt.plot(df_2019_Q1["date"], df_2019_Q1["close"], label="Actual Close Price", marker="o
18         ")
19 plt.plot(df_2019_Q1["date"], df_2019_Q1["predicted_close"], label="Predicted Close
20         Price (Model)", marker="x")
21 plt.title("Actual vs. Predicted Bitcoin Close Price (01/01 - 31/03/2019)")
22 plt.xlabel("Date")
23 plt.ylabel("Close Price (USD)")
24 plt.xticks(rotation=45)
25 plt.legend()
26 plt.grid(True)
27 plt.show()

```



Hình 11: Biểu đồ so sánh giá kết thúc phiên thực tế và dự đoán của Bitcoin (01/01/2019 - 31/03/2019).

Bước 7: Huấn luyện và đánh giá mô hình từ thư viện scikit-learn

```

1 from sklearn.linear_model import LinearRegression
2
3 linear_regressor = LinearRegression()
4 linear_regressor.fit(X_train_scaled, y_train)

```

Mô hình được xây dựng bằng `LinearRegression` của `scikit-learn`, được huấn luyện trên dữ liệu đã chuẩn hoá để tự động tìm ra các hệ số w và bias b tối ưu.

```

1 # Make predictions on the test set
2 y_pred = linear_regressor.predict(X_test_scaled)
3
4 # Evaluation
5 mse = mean_squared_error(y_test, y_pred)
6 mae = mean_absolute_error(y_test, y_pred)
7 r2 = r2_score(y_test, y_pred)
8
9 print("Mean Square Error (MSE):", round(mse, 4))
10 print("Mean Absolute Error (MAE):", round(mae, 4))
11 print("R-squared:", round(r2, 4))

```

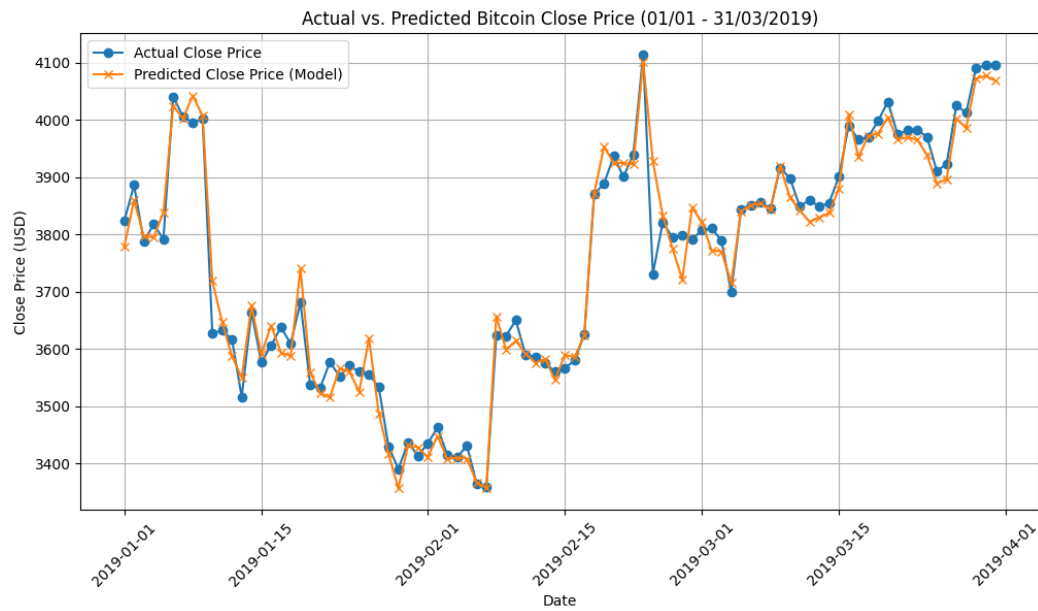
Mô hình sau khi huấn luyện được dùng để dự đoán trên tập kiểm tra, các chỉ số đánh giá gồm MSE (Mean Squared Error), MAE (Mean Absolute Error) và R^2 (R-squared) được tính toán để đo lường chất lượng dự báo.

```

1 # Lọc dữ liệu 2019 Q1
2 df_2019_Q1 = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2019-03-31")].copy()
3
4 # Chọn đúng các cột feature đã train
5 X_2019 = df_2019_Q1[["open", "low", "high"]].values
6 X_2019_scaled = scalar.transform(X_2019)
7 y_2019 = df_2019_Q1["close"].values
8
9 # Dự đoán bằng model sklearn
10 y_pred_2019 = linear_regressor.predict(X_2019_scaled)
11
12 # Gắn vào dataframe để tiện vẽ
13 df_2019_Q1["predicted_close"] = y_pred_2019
14
15 # Vẽ
16 plt.figure(figsize=(12,6))
17 plt.plot(df_2019_Q1["date"], df_2019_Q1["close"], label="Actual Close Price", marker="o")
18 plt.plot(df_2019_Q1["date"], df_2019_Q1["predicted_close"], label="Predicted Close Price (Model)", marker="x")
19 plt.title("Actual vs. Predicted Bitcoin Close Price (01/01 - 31/03/2019)")
20 plt.xlabel("Date")
21 plt.ylabel("Close Price (USD)")
22 plt.xticks(rotation=45)
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Dữ liệu được lọc cho giai đoạn Q1/2019, sau đó chuẩn hoá bằng scaler đã huấn luyện. Mô hình LinearRegression được sử dụng để dự đoán giá đóng cửa, và kết quả được so sánh trực quan với giá thực tế thông qua biểu đồ đường.



Hình 12: Biểu đồ so sánh giá kết thúc phiên thực tế và dự đoán của Bitcoin (dùng sklearn) (01/01/2019 - 31/03/2019).

V. Câu hỏi trắc nghiệm

- (Lý thuyết) Trong Linear Regression, tại sao cần thêm bias b vào vector tham số θ ?
 - Để làm mô hình phức tạp hơn
 - Để dịch chuyển đường hồi quy, tránh ràng buộc đi qua gốc tọa độ
 - Để giảm số chiều của dữ liệu
 - Để tăng tốc độ hội tụ của gradient descent
- (Lý thuyết) So sánh giữa Stochastic Gradient Descent (SGD) và Batch Gradient Descent (BGD), phát biểu nào đúng?
 - SGD dùng toàn bộ dữ liệu trong mỗi bước cập nhật, còn BGD dùng một mẫu duy nhất
 - SGD hội tụ nhanh hơn nhưng dao động nhiều hơn, BGD ổn định hơn nhưng chậm
 - SGD và BGD cho cùng tốc độ hội tụ
 - SGD luôn chính xác hơn BGD
- Với 1 sample ($x = 2, y = 5$), tham số $w = 1, b = 0$. Giá trị dự đoán $\hat{y}$ là gì? Công thức: $\hat{y} = wx + b$
 - 2
 - 3
 - 4
 - 5
- Cho $\hat{y} = 7, y = 4$, hàm mất mát MSE theo 1 mẫu là gì? Công thức: $L = (\hat{y} - y)^2$
 - 3
 - 6
 - 9
 - 12
- Với $\mathbf{x} = \begin{bmatrix} 3 \\ 2 \\ 1 \\ 1 \end{bmatrix}$, $\boldsymbol{\theta} = \begin{bmatrix} 1 \\ 0 \\ 2 \\ 1 \end{bmatrix}$, giá trị dự đoán $\hat{y}$ là gì? Công thức: $\hat{y} = \mathbf{x}^T \boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{x}$
 - 6
 - 8
 - 10
 - 12

6. Cho batch gồm $m = 2$ mẫu: $\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix}$, $\boldsymbol{\theta} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$ Tính vector dự đoán $\hat{\mathbf{y}}$.

Công thức: $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$

- (a) $[4, 3]^\top$
- (b) $[5, 4]^\top$
- (c) $[6, 5]^\top$
- (d) $[3, 2]^\top$

7. Với batch trên, tính hàm mất mát trung bình. Công thức: $L = \frac{1}{m}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$

- (a) 0.5
- (b) 1.0
- (c) 1.5
- (d) 2.0

8. Với sample ($\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$, $y = 5$), $\boldsymbol{\theta} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, tính gradient $\nabla_{\boldsymbol{\theta}} L$. Công thức: $\nabla_{\boldsymbol{\theta}} L = 2(\hat{y} - y)\mathbf{X}^\top$

- (a) $[-2, -4, -2]^\top$
- (b) $[2, 4, 2]^\top$
- (c) $[-4, -8, -4]^\top$
- (d) $[4, 8, 4]^\top$

9. Cho batch $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$, $\boldsymbol{\theta} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$, tính gradient $\nabla_{\boldsymbol{\theta}} L$. Công thức: $\nabla_{\boldsymbol{\theta}} L = \frac{2}{m}\mathbf{X}^\top(\hat{\mathbf{y}} - \mathbf{y})$

- (a) $[0, 0]^\top$
- (b) $[1, 1.5]^\top$
- (c) $[-1, -1.5]^\top$
- (d) $[-2, -3]^\top$

10. Sau 1 bước cập nhật với $\boldsymbol{\theta} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\nabla_{\boldsymbol{\theta}} L = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$, learning rate $\alpha = 0.1$, tham số mới $\boldsymbol{\theta}$ là gì?

Công thức: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} L$

- (a) $[0.8, 0.6]^\top$
- (b) $[1.2, 1.4]^\top$
- (c) $[0.9, 0.7]^\top$
- (d) $[0.7, 0.3]^\top$

Phụ lục

- Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
- Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
- Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 13: Hình ảnh group facebook AIO Q&A

4. Rubric:

Module 4 - Exercise - Linear Regression - Rubric		
Câu	Kiến Thức	Đánh Giá
1	- Khái niệm linear regression và các biến thể của thuật toán Gradient Descent (Stochastic Gradient Descent, Mini Batch Gradient Descent, và Batch Gradient Descent) - Cách thức thực hiện 3 biến thể trên của Gradient Descent theo kiểu Vectorization (với sự hỗ trợ của thư viện numpy) - Cách đọc và xử lý data cơ bản với numpy	- Hiểu và nắm được ý chính cách hoạt động của 3 biến thể của thuật toán Gradient Descent (Stochastic Gradient Descent, Mini Batch Gradient Descent, và Batch Gradient Descent) với Linear Regression - Vận dụng và thao tác với thư viện numpy để thực hiện 3 biến thể trên theo kiểu Vectorization - Biết cách load data kiểu csv và normalize các feature trước khi đi train linear regression model
2	- Hiểu về cách áp dụng của thuật toán Linear Regression cho dữ liệu Time Series - Phân tích một số đặc trưng của dữ liệu Time Series	- Áp dụng Linear Regression cho bài toán dự đoán giá Bitcoin - Đánh giá hiệu quả trên những cài đặt khác nhau